MLFF-DATA9A5: Critical-FP64 MACE Execution

2026-07-29

MLFF-DATA9A5: critical-FP64 MACE execution

1. Purpose

DATA9A5 separates the precision of the expensive MACE body from the precision of numerically sensitive global operations. The user may train and execute the model body in either `float32` or `float64`, while mdstats safety-locks critical reductions, returned observables, and persistent ASE MD state to `float64`.

This stage is intended for consumer GPUs such as the RTX 3090, where a complete FP64 MACE calculation is much slower than FP32 but long MD trajectories still benefit from accurate global reductions and double-precision state updates.

2. Model-body precision

Training precision remains selected by:

```
MaceOptimizerPolicy(default_dtype="float32")
MaceOptimizerPolicy(default_dtype="float64")
```

Python/ASE inference precision is selected independently through `MaceCalculatorProvider.from_model_path(default_dtype=...)` or `build_mace_critical_precision_calculator(model_dtype=...)`.

Only `float32` and `float64` are accepted. The selected training dtype remains part of `TrainingProtocolIdentity`; inference dtype is recorded in the model checkpoint identity.

3. Safety-locked critical precision

`MaceCriticalPrecisionPolicy` requires:

- atomic reference-energy accumulation in FP64;
- interaction-energy accumulation in FP64;
- total-energy output in FP64;
- virial and stress reduction in FP64;
- force, virial, stress, atomic-energy, and global-energy outputs in FP64;
- persistent ASE positions, cell, masses, and momenta in FP64;
- TensorFloat-32 disabled;
- inference forces differentiated from the same FP64-accumulated energy used for the reported potential energy.

The reference and learned interaction energies are reduced separately before being combined. Node-energy reductions use graph-contiguous FP64 segment sums, avoiding an FP32 system-energy reduction.

4. Training autograd boundary

MACE 0.3.16 force training requires second derivatives through an e3nn/MACE force graph. An FP64 scalar seed propagated through a FP32 force-Jacobian path causes an upstream dtype failure. Therefore, optimization-time energy, force, and stress graphs remain entirely in the user-selected model dtype.

This exception is explicit and immutable:

```
training_force_jacobian_dtype = model
```

Validation, standalone evaluation, descriptor/prediction inference, and Python ASE MD use the critical-FP64 path. Thus FP32 training remains supported while production trajectory observables and reductions are double precision.

5. Runtime implementation

The first implementation is version-locked to:

- `mace-torch==0.3.16`;
- `mace.modules.models.ScaleShiftMACE`;
- the Python/ASE calculator execution path.

`install_mace_critical_fp64_patch()` installs the explicit runtime forward adapter. It fails closed for any other MACE version. The mdstats command wrappers install it before real training, evaluation, or head-selection jobs:

- `mdstats-mace-train`;
- `mdstats-mace-eval`;
- `mdstats-mace-select-head`.

LAMMPS, ML-IAP, Kokkos, LibTorch, and accelerator execution are downstream consumer responsibilities. This stage does not inspect, reproduce, or qualify their internal mixed-precision behavior. mdstats supplies a uniformly FP32 or FP64 serialized model and makes no downstream runtime-precision claim.

6. Audits

`MaceCriticalPrecisionAudit` records the MACE version, model class, model-body dtype, global output dtypes, TF32 state, patch state, and policy digest. It passes only when critical outputs are FP64 and TF32 is disabled.

`AseMdStatePrecisionAudit` records positions, cell, masses, and optional momenta. `audit_ase_md_state_precision(..., require_momenta=True)` fails before MD when any persistent state array is not FP64 or when velocities/momenta are absent.

7. Numerical interpretation

Casting final FP32 results to FP64 cannot restore precision lost inside the FP32 equivariant body. DATA9A5 instead prevents additional loss in global energy/virial reduction and repeated MD-state updates. A full FP64 model remains available when internal FP32 force error is scientifically unacceptable.

8. Acceptance requirements

DATA9A5 is accepted only when:

1. FP32 and FP64 training protocols remain independently selectable;
2. FP32 and FP64 Python inference remain independently selectable;
3. TF32 is disabled by the execution policy;
4. a real FP32 MPA-0 calculation returns FP64 energy, force, virial, and stress;
5. the patched FP64 path agrees with upstream full-FP64 MACE to numerical precision;
6. a one-epoch FP32 transfer job completes and the saved model is uniformly FP32;
7. a one-epoch FP64 transfer job remains supported;
8. both saved models pass the post-training critical precision audit;
9. ASE MD-state audits round-trip and fail closed on FP32 momenta;
10. all policy and audit records reject digest tampering.

DATA9A5 does not complete production DATA6-DATA8 realization and does not open DATA9B by itself.